



The application, compiled.

A compiler and runtime that turns React-shaped JSX and TSX into a single native binary: renderer, datastore, live update bus, identity layer and HTTP server, all in one process.

NO NODE.JS IN THE REQUEST PATH

TECHNICAL SPEC SHEET

Rust · SWC · QuickJS · libSQL
MIT licensed

ONE ARTIFACT REPLACES

Node runtime · ORM and migration tool · API or BFF layer · WebSocket service · session store · bundler configuration · client state library · server directory

01 Rendering is decided, not declared

Every component is analysed at build time for what it actually does, then compiled to the cheapest shape that still does it.

Tier A. No hooks, no async, no effects. Plain HTML, and **zero client JavaScript**.

Tier B. Handlers and light interactivity. **Bindings only**: a click updates the bound node in place, with no virtual DOM, no re-render and no network round trip.

Tier C. Full hooks, async and side effects. Conventional client hydration.

The tier is a property of the code, not an annotation. Adding `useState` moves a component down at the next build, and removing it moves the component back. There is no `use client` boundary to declare, drift from, or review.

02 Data: the declaration is the schema

A collection declared in `albedo.config.ts` **exists**. You get a table, seed rows, a live topic and a generated `.d.ts`, with no migration folder, no ORM and no API route.

Field types are `text`, `int`, `real`, `bool` and `timestamp`, with a trailing `?` for nullable. A `bool` round trips as a bool, not as a 1.

Additive evolution is automatic. Add a nullable field and restart: the column appears and existing rows stay intact. Anything narrowing is **refused by name** rather than applied halfway.

`append`, `update` and `remove` are ordinary server actions over an embedded libSQL store.

03 Live is the default, not an add-on

A read mints a **topic**, a write emits a delta, and subscribed clients repaint. The author writes no subscription code and no WebSocket code.

Scalars and lists go through `useSharedSlot`, and updates land **across tabs with no polling**.

Keyed rows. A write repaints the changed row and leaves untouched DOM alone.

Dynamic topics. A route parameter can name a topic, so `/room/[id]`, per-user views and multi-tenancy are ordinary rather than special cases.

Subscription is scoped to the route a client is on, not to every topic in the application.

04 The authoring surface you already have

`.tsx` and `.jsx` parsed with SWC. File-based routing, nested `layout.tsx` composition, dynamic segments, `error.tsx` and `loading.tsx` boundaries, and `async` server components.

`useState`, `useEffect`, `useRef`, `useMemo` and `useCallback`. Global CSS and CSS modules, `<title>` and meta tags.

npm dependencies are bundled in tree for server rendering and for action bodies, so `node_modules` is a build input and never a runtime dependency.

Server rendering and action handlers execute real JavaScript on an embedded engine, so loops, `try` and array methods behave. A broken construct **fails loudly** instead of quietly rendering nothing.

05 Authorization derived from the read

The load bearing idea. Identity is a legal **partition key**, so `posts.where({owner: user.id})` is not a query guarded by a policy. **The read is the policy.**

An author cannot spell a topic name directly, so there is no way to write a query whose authorization disagrees with it, and no policy to keep in sync during review.

Session cookies, first party password credentials under argon2, CSRF on same-origin POST, route gating by declared policy, and rate limiting on credential attempts.

The sign-in flow works with JavaScript disabled.

06 Calling the outside world, under policy

`await fetch(...)` works inside a handler or action body, written as plain JavaScript, exactly the way any vendor's sample spells it. The compiler lowers the `await`, and the server suspends the body then replays it against a journal that answers.

A declared `sources` block makes a remote endpoint a topic. It refreshes, paints like any other slot, and generates its own types.

Egress policy is enforced inside the DNS resolver, so a declared allowlist cannot be walked around by DNS rebinding or by an IP literal URL. Redirects are refused outright.

Response caching, single-flight coalescing, conditional requests, derived idempotency keys and a hard request deadline are properties of the path rather than of your handler.

07 Build, ship, operate

`init` scaffolds a project. `dev` serves it with hot reload. `build` produces the production artifact, `serve` builds and runs it, and `ship` targets a standalone binary or a container image.

`doctor` reports **what the compiler decided and why**: the tier of every component, and the escape analysis underneath it. Bundle size becomes a build output you can diff rather than a code review discipline.

A clean production build of the starter takes **84 ms**, so the compile step stays inside the pause between saving a file and looking at the browser.

One process to supervise, one port, one file to copy. TLS and certificate acquisition are part of the server, not a sidecar.

08 What this buys a CTO

Client JavaScript stops being a judgement call. The compiler decides what ships, not the reviewer, and it decides again on every build.

Row level security stops being reviewable surface. It is structural: unauthorised reads are not caught, they are unwritable.

The integration surface collapses. Fewer moving services means fewer versions to align, fewer boundaries to authenticate across, and one deployment to roll back.

Onboarding is React. Engineers write the components they already write, and the leverage sits in what the build removes rather than in what they have to learn.

09 Measured envelope

starter app, 16 cores, release build, over loopback. p50 of 3,000 samples, every one a 200.

0.06 ms Render, time to first byte
connection reused, uncontended

0.25 ms Render, time to first byte
connection reused, all 16 cores busy

0.23 ms Render, time to first byte
a new connection for every request

0.57 ms First render after a cold boot
measured over ten fresh processes

244 ms Boot to serving
one process, with no Node start-up

567 B Framework bytes on first paint
per route, Tier A, no client JavaScript